Desenvolvimento de Sistemas Web II

Introdução a Go, com testes, e o domínio do serviço Go

DIM0547 — Turma 01 · Sprint 2 · aula de 28/10

Prof. Fernando · UFRN · 2026.2

Sprint 2

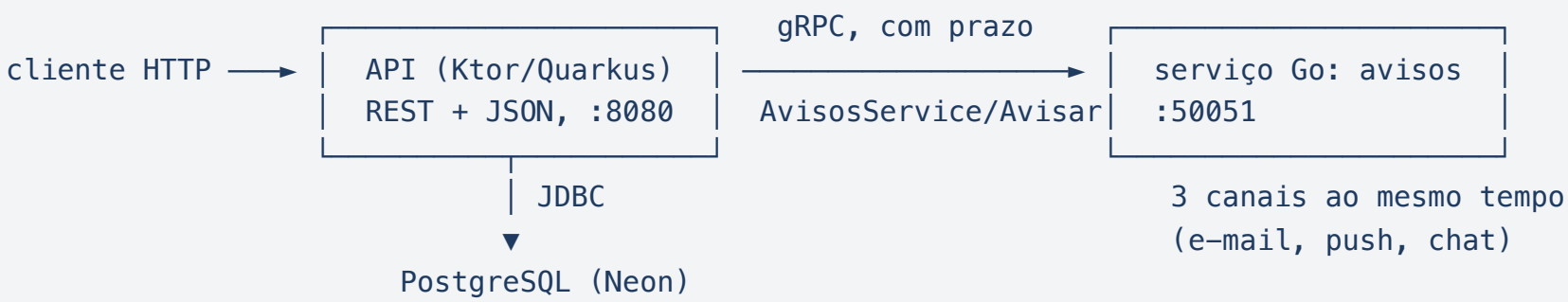
	Data
Conteúdo	hoje: introdução a Go e o Passo 13 · em vídeo, até 30/10: passos 14 a 18
Revisão para a prova	04/11 (presencial) e 09/11 (online)
Acompanhamento online	16/11
Prova escrita	**11/11 (quarta)**, no laboratório
Daily meetings online	23 e 25/11
Entrega final	**30/11, 23:59**

> **Prova (11/11):** presencial, no laboratório, no horário da aula, no **Multiprova**, com questões fechadas sobre todo o conteúdo do curso. É permitida **uma folha A4 manuscrita, frente e verso**; nenhum outro material pode ser consultado.

Critérios da Sprint 2

Critério	Peso	Aula
Microsserviço Go	25%	hoje
Contrato Protobuf e integração gRPC	25%	vídeo
Sistema no ar	25%	vídeo
Ambiente e testes (inclui os testes que saíram da Sprint 1)	15%	vídeo
Documentação	10%	—

Onde vamos chegar



- Ao criar uma tarefa, a API pede ao serviço Go que avise por três canais
- O contrato entre os dois é um arquivo `.proto`, versionado no repositório
- Se o serviço Go cair, a tarefa continua sendo criada

Branch `aula`: `./passo.sh 12` é onde a Sprint 1 terminou; hoje vamos até o 13.

Por que um serviço à parte, e em Go

A rubrica pede um serviço com responsabilidade **justificada**. Bons motivos:

Motivo	No exemplo
Trabalho de E/S que espera por terceiros	cada canal espera um servidor externo
Concorrência	os três canais ao mesmo tempo, uma goroutine por canal
Falha isolada	se o serviço cair, a API continua respondendo
Ritmo próprio de implantação	o serviço muda sem republicar a API

Maus motivos: "microsserviço é moderno"; separar um CRUD em dois serviços que dividem o mesmo banco.

> Go cabe bem aqui: goroutines baratas, binário estático, imagem de container pequena e partida rápida.

📖 **Ref.** Effective Go

Roteiro de hoje

Min	Bloco	Exemplo
10	A Sprint 2 e por que um serviço em Go	a figura do sistema
10	Ferramentas e tipos	`go mod`, `go run`, `go test`
20	Olá, Mundo com testes; funções como valores	`2-hello-world`
15	Métodos e interfaces	`3-metodos-interfaces`
15	Ponteiros e erros	`4-ponteiros-erros`
10	Injeção de dependência	`5-injecao-de-dependencia`
20	Passo 13: o domínio do serviço de avisos	`exemplos/avisos-go`

Os exemplos de Go vêm do repositório do Curso básico de Go, que segue o livro *Aprenda Go com Testes*:

```
git clone https://github.com/classrooms-fmarquesfilho/aprenda-go-com-testes.git
cd aprenda-go-com-testes/2-hello-world && go test -v
```

Como acompanhar sem rodar: as tags

Cada conceito tem um ponto no repositório

Conceito	Onde
Ferramentas, tipos, olá mundo	`aprenda-go-com-testes/2-hello-world`
Métodos e interfaces	`aprenda-go-com-testes/3-metodos-interfaces`
Ponteiros e erros	`aprenda-go-com-testes/4-ponteiros-erros`
Injeção de dependência	`aprenda-go-com-testes/5-injecao-de-dependencia`
O serviço de avisos (Passo 13)	tag `passo-13` no branch `aula` de `web2-2026-2`

No Codespace, `./passo.sh 13` leva ao estado do Passo 13; cada pasta do curso roda com `go test -v`.

Parte 1

Go com testes

As ferramentas

```
go mod init exemplo.com/hello     # cria o módulo: o go.mod, com o nome e a versão do Go
go run .                          # compila e roda o pacote main
go test -v                        # roda os testes (arquivos *_test.go), mostrando cada um
go test -run TestHello/região     # só os testes cujo nome casa com o padrão
gofmt -l .                        # lista os arquivos fora do formato (o formato é um só)
go vet ./...                      # aponta erros comuns que compilam
```

Termo	O que é
módulo	o projeto: uma pasta com `go.mod`, que declara o nome e as dependências
pacote	uma pasta de arquivos `.go` com o mesmo `package`; `main` gera um executável
teste	uma função `TestXxx(t *testing.T)` num arquivo `_test.go`, ao lado do código

O teste vem com a linguagem: não há biblioteca de teste para escolher nem configurar.

Tipos básicos e o valor zero

Tipo	Exemplo	Valor zero
`bool`	`true`	`false`
`string`	`"oi"`	`""`
`int`, `int64`	`42`	`0`
`float64`	`3.14`	`0`
`byte` / `rune`	`'a'`, `'ç'`	`0`

- Não há valor "não inicializado": toda variável nasce com o **valor zero** do seu tipo.
- `rune` é um `int32` que representa um caractere Unicode; `string` é uma sequência de bytes.
- `int` tem o tamanho da arquitetura; `int32` / `int64` têm tamanho fixo.

📖 **Ref.** A Tour of Go — básico · Go spec — tipos

Tipagem: estática, forte e com inferência

```go
var nome string = "Maria"      // tipo declarado
idade := 20                    // tipo inferido (int), só dentro de funções
const HourNotSpecified = -1    // constante

total := idade + 2.5           // erro de compilação: int + float64
total := float64(idade) + 2.5 // conversão sempre explícita
```

	Go	Java	Python
Quando os tipos são verificados	compilação (estática)	compilação	execução (dinâmica)
Conversão implícita entre tipos	não (forte)	em alguns casos (`int` → `double`)	não, mas os tipos só aparecem na execução
Precisa escrever o tipo?	não, quando o compilador deduz	`var` desde o Java 10	não

> Inferência não é tipagem dinâmica: `idade := 20` tem tipo `int` para sempre.

📖 **Ref.** Curso básico de Go, `2-hello-world/slides/slides-tipos-em-go.md`

Variáveis, constantes e `:=`

```go
var nome string = "Maria"   // declaração explícita
idade := 20                  // := declara e infere, só dentro de função
const horas = 24             // constante: não muda

var (                        // bloco de declarações
    x, y int
    ok  bool
)
```

- `:=` é uma **declaração curta**: cria a variável e infere o tipo; não vale no escopo de pacote.
- `const` vale em qualquer escopo; o valor é conhecido em compilação.
- Variável declarada e não usada (dentro de função) é **erro de compilação**.

📖 **Ref.** A Tour of Go — variáveis · Effective Go — short declarations

Olá, Mundo, com o teste primeiro

O ciclo do livro: **vermelho** (um teste que falha) → **verde** (o mínimo para passar) → **refatorar** (melhorar com os testes verdes).

```go
func assertMessage(t testing.TB, got, want string) {
    t.Helper()                      // a falha aponta a linha de quem chamou
    if got != want {
        t.Errorf("got %q want %q", got, want)
    }
}

func TestHello(t *testing.T) {
    t.Run("sem opções", func(t *testing.T) {          // subteste com nome
        assertMessage(t, Hello(), "Olá, Mundo")
    })
    t.Run("nome e hora", func(t *testing.T) {
        assertMessage(t, Hello(WithName("Maria"), WithHour(9)), "Bom dia, Maria")
    })
    t.Run("hora 24 - inválida", func(t *testing.T) {   // os limites também são casos
        assertMessage(t, Hello(WithHour(24)), "Olá, Mundo")
    })
}
```

Na tela: `cd 2-hello-world && go test -v`; quebrar um caso e ler a mensagem; `go test -run 'TestHello/região'`.

Um laço só: o `for`

```
for i := 0; i < 3; i++ { ... }      // o for "clássico"
for _, t := range tarefas { ... }   // percorrer uma fatia/mapa
for condicao { ... }                // o "while" é um for sem init/pós
for { ... }                         // loop infinito
```

- Go tem **um único** laço, o `for`, em três formas.
- `range` devolve índice e valor; `_` descarta o que não interessa.
- `defer` adia uma chamada para o fim da função, em ordem LIFO: é o "garantir que fecha".

📖 **Ref.** A Tour of Go — for · Effective Go — defer

Funções como valores: *functional options*

```go
type HelloOption func(*HelloConfig)          // um tipo que é uma função

func WithName(name string) HelloOption {
    return func(cfg *HelloConfig) { cfg.Name = name }   // devolve uma função que lembra de name
}

func Hello(options ...HelloOption) string {  // variádica: zero ou mais opções
    cfg := HelloConfig{Hour: HourNotSpecified}
    for _, option := range options {
        option(&cfg)                         // cada opção altera a configuração
    }
    ...
}

Hello()
Hello(WithName("João"), WithHour(20), WithRegion(RegionRN))
```

- Go não tem sobrecarga nem parâmetros com valor padrão: as opções resolvem as duas coisas
- `WithName` devolve uma *closure*: uma função que guarda o `name` recebido
- `&cfg` passa o endereço da configuração; é por ele que a opção consegue alterá-la (mais adiante, ponteiros)

📖 **Ref.** Curso básico de Go, `3-metodos-interfaces/slides/polimorfismo-em-go.md` (Overloading × Functional Options)

Funções: retorno múltiplo e erro por último

```go
func Divisao(a, b int) (int, error) {   // o erro vem por último, por convenção
    if b == 0 {
        return 0, fmt.Errorf("divisor zero")
    }
    return a / b, nil
}

q, err := Divisao(10, 2)
if err != nil { ... }                   // sempre verificar o erro
```

- Uma função devolve vários valores; o par `(resultado, error)` é o idioma do Go.
- O chamador **deve** tratar o erro — o compilador não obriga, mas o estilo sim.
- *Named return* (`func f() (q int, err error)`) nomeia os retornos e permite `return` "nu".

📖 **Ref.** A Tour of Go — funções · Effective Go — multiple returns

Métodos e interfaces: satisfação implícita

```
type Forma interface {
    Area() float64
}

type Retangulo struct {
    Largura float64
    Altura  float64
}

func (r Retangulo) Area() float64 {          // um método: a função tem um receptor
    return r.Largura * r.Altura
}

type Circulo struct{ Raio float64 }

func (c Circulo) Area() float64 { return math.Pi * c.Raio * c.Raio }
```

```
func checarArea(t testing.TB, forma Forma, want float64) { ... }   // aceita qualquer Forma

checarArea(t, Retangulo{Largura: 12, Altura: 6}, 72.0)
checarArea(t, Circulo{Raio: 10}, 314.1592653589793)
```

Uma `interface` é um conjunto de métodos que um tipo oferece; `Forma` exige o método `Area() float64`. Um método é uma função presa a um tipo pelo receptor entre parênteses.

Nenhum `implements`: quem tem o método `Area() float64` é uma `Forma`, e o compilador confere.

`struct`: agregação, sem herança

```
type Entrega struct {
    Canal string
    Erro  error
}

type Avisador struct {          // "herança" não existe: compõe-se
    canais []Canal
}
```

- Go não tem classes nem herança: reúna dados numa `struct` e associe **métodos**.
- Campo com letra minúscula é privado do pacote; maiúscula, exportado.
- **Embutimento** (embedding) promove os campos/métodos do tipo embutido, sem hierarquia.

📖 **Ref.** A Tour of Go — structs · Effective Go — embedding

Interfaces em Java e em Go

Java

```java
interface Forma { double area(); }

class Retangulo implements Forma {
    public double area() { ... }
}
```

A classe declara que implementa. A interface precisa existir antes, e a classe depende dela.

Go

```go
type Forma interface { Area() float64 }

type Retangulo struct{ ... }

func (r Retangulo) Area() float64 { ... }
```

O tipo não sabe da interface. Quem usa declara a interface de que precisa.

Consequência: em Go, a interface fica **com quem usa**, e costuma ter um ou dois métodos. É o que vamos ver em `io.Writer` e no serviço de avisos (`Canal`).

Go × Java: quatro trocas de mentalidade

Java	Go
herança e `implements`	interface satisfeita implicitamente
exceções (`throw` / `catch`)	erro é um valor devolvido
`Thread` e executor	goroutine (leve, barata)
classes e construtores	`struct` e funções

Não é questão de "melhor": são decisões de design do Go, e é preciso aceitá-las para escrever código idiomático.

📖 **Ref.** Effective Go · pastas `3-metodos-interfaces` e `4-ponteiros-erros`

Ponteiros: o receptor por valor recebe uma cópia

```
type Carteira struct {
    saldo int                                   // minúscula: invisível fora do pacote
}

func (c *Carteira) Depositar(quantia int) {        // *Carteira: o método altera a carteira original
    c.saldo += quantia
}
```

O que acontece sem o `*`:

```
func (c Carteira) Depositar(quantia int) { c.saldo += quantia }   // c é uma cópia

carteira := Carteira{}
carteira.Depositar(10)
carteira.Saldo()          // 0: o depósito foi feito na cópia
```

Receptor	Quando usar
por valor `(c Carteira)`	o método só lê; o tipo é pequeno
por ponteiro `(c *Carteira)`	o método altera o estado, ou o tipo é grande

Na tela: em `4-ponteiros-erros`, tirar o `*` de `Depositar` e rodar `go test -v`.

Erros: valores, com tipo próprio

```go
type ErroSaldoInsuficiente struct{ Solicitado, Disponivel int }

func (e ErroSaldoInsuficiente) Error() string {   // quem tem Error() string é um error
    return fmt.Sprintf("não é possível sacar %d, pois o saldo disponível é %d", e.Solicitado, e.Disponivel)
}

func (c *Carteira) Sacar(quantia int) error {
    if quantia > c.saldo {
        return ErroSaldoInsuficiente{Solicitado: quantia, Disponivel: c.saldo}
    }
    c.saldo -= quantia
    return nil                                    // nil: deu certo
}
```

```go
if err := carteira.Sacar(10); err != nil {
    var falta ErroSaldoInsuficiente
    if errors.As(err, &falta) {                   // recupera o erro com os campos
        fmt.Println("faltam", falta.Solicitado-falta.Disponivel)
    }
}
```

- `error` é uma interface com um método, `Error() string`; o erro aparece na assinatura e é tratado por quem chamou

📖 **Ref.** Go — errors

Erros: embrulhar e reconhecer

```
var ErrSaldoInsuficiente = errors.New("saldo insuficiente")   // erro sentinela

return fmt.Errorf("sacar %d: %w", quantia, ErrSaldoInsuficiente)   // %w embrulha

if errors.Is(err, ErrSaldoInsuficiente) { ... }   // reconhece pela identidade
var falta ErroSaldoInsuficiente
if errors.As(err, &falta) { ... }                 // recupera o tipo com dados
```

- `%w` preserva a cadeia de erros; `%v` não.
- **`errors.Is`** compara, na cadeia, com um erro sentinela; **`errors.As`** recupera um tipo de erro.
- Regra: erro com nome quando basta saber qual; tipo quando o chamador precisa dos dados.

📖 **Ref.** Go — errors · Effective Go — errors

Injeção de dependência: `io.Writer`

```go
type Writer interface { Write(p []byte) (n int, err error) }   // io.Writer: um método só

func Saudar(escritor io.Writer, options ...HelloOption) {
    fmt.Fprintf(escritor, "%s", Hello(options...))
}
```

Quem chama passa	E a saudação vai para
`os.Stdout`	o terminal
`&bytes.Buffer{}`	a memória, para o teste conferir
o `http.ResponseWriter` de um servidor HTTP	a resposta da requisição

```go
func TestSaudar(t *testing.T) {
    buffer := bytes.Buffer{}
    Saudar(&buffer, WithName("Maria"))
    if buffer.String() != "Olá, Maria" { t.Errorf(...) }
}
```

Depender de uma interface, e não de um destino concreto, é o que torna a função testável: a ideia das portas da API de Tarefas.

📖 **Ref.** Curso básico de Go, `5-injecao-de-dependencia/slides/5-injecao-dependencia-slides.md`

Testes: *table-driven* e subtestes

```go
func TestHello(t *testing.T) {
    casos := []struct{ nome string; opcoes []HelloOption; quer string }{
        {"sem opções", nil, "Olá, Mundo"},
        {"com nome", []HelloOption{WithName("Maria")}, "Olá, Maria"},
    }
    for _, c := range casos {
        t.Run(c.nome, func(t *testing.T) {       // cada caso vira um subteste
            if got := Hello(c.opcoes...); got != c.quer {
                t.Errorf("got %q want %q", got, c.quer)
            }
        })
    }
}
```

- **Table-driven**: uma tabela de casos, um `for` que os percorre — a forma idiomática de testar.
- `t.Run` dá nome a cada caso e permite rodar um só (`go test -run 'TestHello/com nome'`).
- Testes vivem ao lado do código, em arquivos `_test.go`.

📖 **Ref.** Go — testing · Effective Go — table driven

Parte 2 — Passo 13

O domínio do serviço de avisos

O domínio: uma interface pequena e o Avisador

```
// Canal é qualquer meio de entrega: e-mail, push, chat. A interface é pequena e
// declarada aqui, do lado de quem usa, como é costume em Go.
type Canal interface {
    Nome() string
    Enviar(ctx context.Context, aviso Aviso) error
}

type Avisador struct {
    canais []Canal
}

func NovoAvisador(canais ...Canal) *Avisador {        // "construtor": uma função comum
    return &Avisador{canais: canais}
}
```

- `CanalSimulado` tem `Nome()` e `Enviar(...)`: é um `Canal`, sem declarar nada
- O pacote `avisos` é o domínio: não importa gRPC, HTTP nem banco
- Como o `io.Writer`: uma interface pequena, declarada por quem usa, com um método que pode falhar

Concorrência: uma goroutine por canal

```go
func (av *Avisador) Avisar(ctx context.Context, aviso Aviso) ([]Entrega, error) {
    if aviso.TarefaID <= 0 || strings.TrimSpace(aviso.Titulo) == "" {
        return nil, fmt.Errorf("%w: tarefa %d, título %q", ErrAvisoInvalido, aviso.TarefaID, aviso.Titulo)
    }
    entregas := make([]Entrega, len(av.canais))
    var wg sync.WaitGroup
    for i, canal := range av.canais {
        wg.Go(func() {              // dispara uma goroutine e a registra no WaitGroup
            entregas[i] = Entrega{Canal: canal.Nome(), Erro: canal.Enviar(ctx, aviso)}
        })
    }
    wg.Wait()                       // espera todas terminarem
    return entregas, nil
}
```

- O `sync.WaitGroup` conta as goroutines disparadas: `Go` registra uma, e `Wait` bloqueia até todas terminarem
- Cada goroutine escreve só na sua posição da fatia: não há disputa
- Canais de 400, 50 e 150 ms: em sequência, 600 ms; ao mesmo tempo, **400 ms**, o do mais lento

📖 **Ref.** Go — sync.WaitGroup

Canais: comunicar compartilhando

```
c := make(chan int)      // canal sem buffer: o envio espera quem receba

go func() { c <- 42 }()  // envia
v := <-c                 // recebe
```

- "Não comunique compartilhando memória; compartilhe memória comunicando" — o modelo de concorrência do Go.
- Canais passam valores **e** sincronizam: o envio/recebimento é um ponto de encontro.
- No exemplo, o cancelamento não usa channel próprio: `ctx.Done()` já é um canal, e o `select` espera ele ou o `time.After`.

📖 **Ref.** A Tour of Go — channels · Effective Go — channels

Goroutine não é thread

- Uma goroutine é uma linha de execução **leve**: o runtime agrupa milhares delas em poucas threads do sistema.
- Criar uma custa quase nada: `go f()`.
- Por isso o exemplo dispara uma goroutine por canal sem peso na consciência.

> Regra de ouro: **não comunique compartilhando memória; compartilhe memória comunicando** (via canais).

📖 **Ref.** A Tour of Go — Concurrency · tag `passo-13`

`context`: prazo e cancelamento

```go
func (c CanalSimulado) Enviar(ctx context.Context, aviso Aviso) error {
    select {                                  // espera o que acontecer primeiro
    case <-time.After(c.demora):              // o canal terminou
        return nil
    case <-ctx.Done():                        // o prazo venceu ou alguém cancelou
        return fmt.Errorf("canal %s: %w", c.nome, ctx.Err())
    }
}
```

```go
ctx, cancelar := context.WithTimeout(context.Background(), 100*time.Millisecond)
defer cancelar()                              // defer: roda ao sair da função
entregas, err := av.Avisar(ctx, aviso)       // push (50 ms) entrega; email e chat desistem
```

- Toda função que pode esperar recebe `ctx context.Context` como **primeiro** parâmetro
- O prazo atravessa as camadas: quem chama decide quanto está disposto a esperar

📖 **Ref.** Go — context

`context`: o prazo atravessa as camadas

- Quem chama decide **quanto está disposto a esperar** e põe isso no `context`.
- Quem é chamado **respeita**: consulta `ctx.Done()` e desiste quando o prazo vence.
- Isso vale na memória e, no Passo 15, **atravessa a rede** (o prazo do gRPC vira o prazo do `context`).

> No Passo 13, `go run ./cmd/demo` mostra o prazo de 1 s (três entregas) e o de 100 ms (só o push). Confiram na tag `passo-13`.

📖 **Ref.** Go — context

Erros com nome, e a demonstração

```
var ErrAvisoInvalido = errors.New("aviso inválido")           // um erro com nome

return nil, fmt.Errorf("%w: tarefa %d", ErrAvisoInvalido, id) // %w embrulha, sem perder a origem

if errors.Is(err, avisos.ErrAvisoInvalido) { ... }            // compara pela identidade, não pelo texto
```

	Quando usar	Como conferir
erro com nome (`ErrAvisoInvalido`)	quem chama só precisa saber **qual** erro foi	`errors.Is(err, ErrAvisoInvalido)`
tipo de erro (`ErroSaldoInsuficiente`)	quem chama precisa de **dados** do erro	`errors.As(err, &alvo)`

Demonstração

```
cd exemplos/avisos-go
go test ./...          # 3 testes do domínio
go run ./cmd/demo      # prazo de 1 s: 3 entregues em 400 ms; prazo de 100 ms: só o push
```

📖 **Ref.** Go — errors

No vídeo (até 30/10)

Passo	Assunto
14	O contrato: Protocol Buffers e Buf
15	O servidor gRPC em Go: status, prazo, health e reflection
16	A API chama o Go: cliente gRPC no Ktor e no Quarkus
17	Tudo em containers: Dockerfiles, health checks e `docker compose`
18	Implantação: imagens no GHCR, migração por comando, Render e Neon

Até lá, no projeto do grupo:

- fazer os exercícios do curso de Go que faltarem (`2-hello-world` a `5-injecao-de-dependencia`)
- escolher a responsabilidade do serviço Go e escrever a justificativa (uma ADR)
- escrever o domínio do serviço, com testes, como no Passo 13

Onde estudar depois

- Curso básico de Go (vídeos do professor) e o repositório dos exemplos
- Aprenda Go com Testes: os capítulos de Olá, Mundo; estruturas, métodos e interfaces; ponteiros e erros; injeção de dependência; concorrência e select
- Learn Go with Tests — Context (em inglês)
- A Tour of Go · Effective Go
- Leitura da sprint: `leituras/web2-s2.md`, capítulos 1 a 3
- Passo a passo: `exemplos/avisos-go/PASSOS.md` · branch `aula`, tag `passo-13`